

Started on	Sunday, 8 June 2025, 10:40 PM
State	Finished
Completed on	Thursday, 12 June 2025, 8:15 AM
Time taken	3 days 9 hours
Marks	3.90/4.00
Grade	9.75 out of 10.00 (98%)

Cho một đồ thị **vô hướng đơn**. Hãy dựng (các) cây DUYỆT ĐỒ THỊ khi duyệt đồ thị theo chiều rộng bắt đầu từ đỉnh 1.

Nếu vẫn còn đỉnh chưa duyệt sau khi duyệt xong lần 1, tìm đỉnh có chỉ số nhỏ nhất chưa duyệt mà duyệt nó, và cứ tiếp tục như thế cho đến khi tất cả các đỉnh đều được duyệt.

Quy ước:

- Các đỉnh kề của 1 đỉnh được liệt kê *theo thứ tự tăng dần*

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v) .

Đầu ra (Output)

- In ra cây duyệt đồ thị theo định dạng:

```
1 <đỉnh cha của 1>
2 <đỉnh cha của 2>
...
i <đỉnh cha của i>
...
n <đỉnh cha của n>
```

[Copy to BlackBox](#)

- Nếu 1 đỉnh không có đỉnh cha (nó là đỉnh gốc của cây) thì đỉnh cha của nó là -1.
- Xem thêm các ví dụ bên dưới.

Gợi ý

- Sử dụng mảng `parent[u]` để lưu đỉnh cha của đỉnh u .
- Trong quá trình duyệt, thay vì in các đỉnh ra màn hình, ghi nhận lại đỉnh cha của các đỉnh.
- Khi duyệt xong lần lượt in ra u và `parent[u]` (u chạy từ 1 đến n).

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **`freopen("dt.txt", "r", stdin);`** vào ngay sau hàm `main()`. Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

[Copy to BlackBox](#)

For example:

Input	Result
4 3	1 -1
2 1	2 1
1 4	3 -1
2 4	4 1

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4
5 //-----Queue-----
6 typedef struct {
7     int data[MAX_SIZE];
8     int front, rear;
9 } Queue;
```

```

9     Queue,
10
11 void make_null_queue(Queue* pQ) {
12     pQ->rear = -1;
13     pQ->front = 0;
14 }
15
16 int is_empty(Queue* pQ) {
17     return pQ->front > pQ->rear;
18 }
19
20 void enqueue(Queue* pQ, int x) {
21     pQ->rear++;
22     pQ->data[pQ->rear] = x;
23 }
24
25 void dequeue(Queue* pQ) {
26     pQ->front++;
27 }
28
29 int front(Queue* pQ) {
30     return pQ->data[pQ->front];
31 }
32
33 //-----Graph-----
34 typedef struct {
35     int A[MAX_SIZE][MAX_SIZE];
36     int n, m;
37 } Graph;
38
39 // Create empty graph
40 void init_graph(Graph* pG, int n) {
41     pG->n = n;
42     for (int i = 0; i <= pG->n; i++) {
43         for (int j = 0; j <= pG->n; j++) {
44             pG->A[i][j] = 0;
45         }
46     }
47 }
48
49 // Add new edge to graph
50 void add_edge(Graph* pG, int u, int v) {
51     pG->A[u][v] = 1;
52     pG->A[v][u] = 1;
53     pG->m++;
54 }
55
56 //-----BFS-----
57 int visited[MAX_SIZE];
58 int parent[MAX_SIZE];
59
60 void BFS(Graph* pG, int s) {
61     Queue Q;
62     make_null_queue(&Q);
63     enqueue(&Q, s);
64     visited[s] = 1;
65     parent[s] = -1;
66     while (!is_empty(&Q)) {
67         int u = front(&Q);
68         dequeue(&Q);
69         // printf("%d\n", u);
70         for (int v = 1; v <= pG->n; v++) {
71             if (pG->A[u][v] > 0 && !visited[v]) {
72                 enqueue(&Q, v);
73                 parent[v] = u;
74                 visited[v] = 1;
75             }
76         }
77     }
78 }
79
80 int main() {
81     Graph G;
82     int n, m;
83     scanf("%d%d", &n, &m);
84     init_graph(&G, n);
85
86     for (int e = 0; e < m; e++) {
87         int u, v;

```

```

87     int u, v;
88     scanf("%d %d", &u, &v);
89     add_edge(&G, u, v);
90 }
91
92 for (int e = 0; e <= n; e++) {
93     visited[e] = 0;
94     parent[e] = 99;
95 }
96
97 for (int e = 1; e <= n; e++) {
98     if (!visited[e]) {
99         BFS(&G, e);
100     }
101 }
102
103 for (int e = 1; e <= n; e++) {
104     printf("%d %d\n", e, parent[e]);
105 }
106 }
107

```

	Input	Expected	Got	
✓	4 3 2 1 1 4 2 4	1 -1 2 1 3 -1 4 1	1 -1 2 1 3 -1 4 1	✓
✓	4 2 1 2 3 4	1 -1 2 1 3 -1 4 3	1 -1 2 1 3 -1 4 3	✓
✓	4 2 1 4 2 3	1 -1 2 -1 3 2 4 1	1 -1 2 -1 3 2 4 1	✓
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 6 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	1 -1 2 1 3 7 4 1 5 6 6 4 7 4 8 5 9 5 10 12 11 12 12 1 13 6	1 -1 2 1 3 7 4 1 5 6 6 4 7 4 8 5 9 5 10 12 11 12 12 1 13 6	✓

Passed all tests! ✓

Question author's solution (C):

```

1 #include <stdio.h>
2
3 /* Khai báo CSDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;

```

```

13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30 /* Khai báo CTDL Queue */
31
32 #define MAX_SIZE 100
33 typedef struct {
34     int u, p;
35 } ElementType;
36
37 typedef struct {
38     ElementType data[MAX_SIZE];
39     int front, rear;
40 } Queue;
41
42 /* Khởi tạo hàng đợi rỗng */
43 void make_null_queue(Queue *pQ) {
44     pQ->front = 0;
45     pQ->rear = -1;
46 }
47 /* Đưa phần tử u vào cuối hàng đợi */
48 void enqueue(Queue *pQ, ElementType u) {
49     pQ->rear++;
50     pQ->data[pQ->rear] = u;
51 }
52
53 /* Xem phần tử đầu hàng đợi */
54 ElementType front(Queue *pQ) {
55     return pQ->data[pQ->front];
56 }
57
58 /* Xóa bỏ phần tử đầu hàng đợi */
59 void dequeue(Queue *pQ) {
60     pQ->front++;
61 }
62
63 /* Kiểm tra hàng đợi rỗng */
64 int empty(Queue *pQ) {
65     return pQ->front > pQ->rear;
66 }
67
68
69 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
70 int mark[MAX_N];
71 int parent[MAX_N];
72
73 void BFS(Graph *pG, int s) {
74     //1. Khai báo hàng đợi Q, khởi tạo rỗng
75     Queue Q;
76     make_null_queue(&Q);
77     //2. Đưa s vào Q, bắt đầu duyệt từ đỉnh s */
78     ElementType pair;
79     pair.u = s; pair.p = -1;
80     enqueue(&Q, pair);
81     //3. Vòng lặp chính dùng để duyệt
82     while (!empty(&Q)) {
83         //3a. Lấy phần tử ở đầu hàng đợi
84         ElementType pair = front(&Q); dequeue(&Q);
85         int u = pair.u, p = pair.p;
86         if (mark[u] != 0) //u đã duyệt rồi, bỏ qua
87             continue;
88
89         //printf("Duyet %d\n", u); //Làm gì đó trên u
90         mark[u] = 1; //Đánh dấu nó đã duyệt

```

```

91     mark[u] = 1; //Đánh dấu nó đã duyệt
92     parent[u] = p;
93     //3b. Xét các đỉnh kề của u, đưa vào hàng đợi Q
94     for (int v = 1; v <= pG->n; v++)
95         if (adjacent(pG, u, v)) {
96             ElementType pair;
97             pair.u = v; pair.p = u;
98             enqueue(&Q, pair);
99         }
100     }
101 }
102
103
104
105 int main() {
106     //1. Khai báo đồ thị G
107     Graph G;
108     //2. Đọc dữ liệu và dựng đồ thị
109     int n, m, u, v;
110     scanf("%d%d", &n, &m);
111     init_graph(&G, n);
112     for (int e = 0; e < m; e++) {
113         scanf("%d%d", &u, &v);
114         add_edge(&G, u, v);
115     }
116
117     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
118     for (int u = 1; u <= G.n; u++) {
119         mark[u] = 0;
120         parent[u] = -1;
121     }
122
123     //4. Gọi hàm DFS
124     for (int u = 1; u <= G.n; u++)
125         if (mark[u] == 0)
126             BFS(&G, u);
127
128
129     for (int u = 1; u <= G.n; u++)
130         printf("%d %d\n", u, parent[u]);
131
132
133     return 0;
134 }
135
136

```

Correct

Marks for this submission: 1.00/1.00.

Cho một đồ thị **có hướng đơn**. Hãy dựng (các) cây DUYỆT ĐỒ THỊ khi duyệt đồ thị theo chiều rộng bắt đầu từ đỉnh 1.

Nếu vẫn còn đỉnh chưa duyệt sau khi duyệt xong lần 1, tìm đỉnh có chỉ số nhỏ nhất chưa duyệt mà duyệt nó, và cứ tiếp tục như thế cho đến khi tất cả các đỉnh đều được duyệt.

Quy ước:

- Các đỉnh kề của 1 đỉnh được liệt kê *theo thứ tự tăng dần*

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v) .

Đầu ra (Output)

- In ra cây duyệt đồ thị theo định dạng:

```
1 <đỉnh cha của 1>
2 <đỉnh cha của 2>
...
i <đỉnh cha của i>
..
n <đỉnh cha của n>
```

[Copy to BlackBox](#)

- Nếu 1 đỉnh không có đỉnh cha (nó là đỉnh gốc của cây) thì đỉnh cha của nó là -1.
- Xem thêm các ví dụ bên dưới.

Gợi ý

- Sử dụng mảng `parent[u]` để lưu đỉnh cha của đỉnh u .
- Trong quá trình duyệt, thay vì in các đỉnh ra màn hình, ghi nhận lại đỉnh cha của các đỉnh.
- Khi duyệt xong lần lượt in ra u và `parent[u]` (u chạy từ 1 đến n).

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **`freopen("dt.txt", "r", stdin);`** vào ngay sau hàm `main()`. Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

[Copy to BlackBox](#)

For example:

Input	Result
4 3	1 -1
2 1	2 -1
1 4	3 -1
2 4	4 1

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4
5 //-----Queue-----
6 typedef struct {
7     int data[MAX_SIZE];
8     int front, rear;
9 } Queue;
```



```

9 } Queue;
10
11 void make_null_queue(Queue* pQ) {
12     pQ->rear = -1;
13     pQ->front = 0;
14 }
15
16 int is_empty(Queue* pQ) {
17     return pQ->front > pQ->rear;
18 }
19
20 void enqueue(Queue* pQ, int x) {
21     pQ->rear++;
22     pQ->data[pQ->rear] = x;
23 }
24
25 void dequeue(Queue* pQ) {
26     pQ->front++;
27 }
28
29 int front(Queue* pQ) {
30     return pQ->data[pQ->front];
31 }
32
33 //-----Graph-----
34 typedef struct {
35     int A[MAX_SIZE][MAX_SIZE];
36     int n, m;
37 } Graph;
38
39 // Create empty graph
40 void init_graph(Graph* pG, int n) {
41     pG->n = n;
42     for (int i = 0; i <= pG->n; i++) {
43         for (int j = 0; j <= pG->n; j++) {
44             pG->A[i][j] = 0;
45         }
46     }
47 }
48
49 // Add new edge to graph
50 void add_edge(Graph* pG, int u, int v) {
51     pG->A[u][v] = 1;
52     // pG->A[v][u] = 1;
53     pG->m++;
54 }
55
56 //-----BFS-----
57 int visited[MAX_SIZE];
58 int parent[MAX_SIZE];
59
60 void BFS(Graph* pG, int s) {
61     Queue Q;
62     make_null_queue(&Q);
63     enqueue(&Q, s);
64     visited[s] = 1;
65     parent[s] = -1;
66     while (!is_empty(&Q)) {
67         int u = front(&Q);
68         dequeue(&Q);
69         // printf("%d\n", u);
70         for (int v = 1; v <= pG->n; v++) {
71             if (pG->A[u][v] > 0 && !visited[v]) {
72                 enqueue(&Q, v);
73                 parent[v] = u;
74                 visited[v] = 1;
75             }
76         }
77     }
78 }
79
80 int main() {
81     Graph G;
82     int n, m;
83     scanf("%d%d", &n, &m);
84     init_graph(&G, n);
85
86     for (int e = 0; e < m; e++) {
87         int u, v;

```

```

87     int u, v;
88     scanf("%d %d", &u, &v);
89     add_edge(&G, u, v);
90 }
91
92 for (int e = 0; e <= n; e++) {
93     visited[e] = 0;
94     parent[e] = 99;
95 }
96
97 for (int e = 1; e <= n; e++) {
98     if (!visited[e]) {
99         BFS(&G, e);
100     }
101 }
102
103 for (int e = 1; e <= n; e++) {
104     printf("%d %d\n", e, parent[e]);
105 }
106 }
107

```

	Input	Expected	Got	
✓	4 3 2 1 1 4 2 4	1 -1 2 -1 3 -1 4 1	1 -1 2 -1 3 -1 4 1	✓
✓	4 2 1 2 3 4	1 -1 2 1 3 -1 4 3	1 -1 2 1 3 -1 4 3	✓
✓	4 2 1 4 2 3	1 -1 2 -1 3 2 4 1	1 -1 2 -1 3 2 4 1	✓
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 6 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	1 -1 2 1 3 -1 4 1 5 -1 6 4 7 4 8 5 9 5 10 -1 11 10 12 1 13 6	1 -1 2 1 3 -1 4 1 5 -1 6 4 7 4 8 5 9 5 10 -1 11 10 12 1 13 6	✓

Passed all tests! ✓

Question author's solution (C):

```

1 #include <stdio.h>
2
3 /* Khai báo CTDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;

```

```

12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     //if (u != v)
21     //    pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30 /* Khai báo CTDL Queue */
31
32 #define MAX_SIZE 100
33 typedef struct {
34     int u, p;
35 } ElementType;
36
37 typedef struct {
38     ElementType data[MAX_SIZE];
39     int front, rear;
40 } Queue;
41
42 /* Khởi tạo hàng đợi rỗng */
43 void make_null_queue(Queue *pQ) {
44     pQ->front = 0;
45     pQ->rear = -1;
46 }
47 /* Đưa phần tử u vào cuối hàng đợi */
48 void enqueue(Queue *pQ, ElementType u) {
49     pQ->rear++;
50     pQ->data[pQ->rear] = u;
51 }
52
53 /* Xem phần tử đầu hàng đợi */
54 ElementType front(Queue *pQ) {
55     return pQ->data[pQ->front];
56 }
57
58 /* Xóa bỏ phần tử đầu hàng đợi */
59 void dequeue(Queue *pQ) {
60     pQ->front++;
61 }
62
63 /* Kiểm tra hàng đợi rỗng */
64 int empty(Queue *pQ) {
65     return pQ->front > pQ->rear;
66 }
67
68
69 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
70 int mark[MAX_N];
71 int parent[MAX_N];
72
73 void BFS(Graph *pG, int s) {
74     //1. Khai báo hàng đợi Q, khởi tạo rỗng
75     Queue Q;
76     make_null_queue(&Q);
77     //2. Đưa s vào Q, bắt đầu duyệt từ đỉnh s */
78     ElementType pair;
79     pair.u = s; pair.p = -1;
80     enqueue(&Q, pair);
81     //3. Vòng lặp chính dùng để duyệt
82     while (!empty(&Q)) {
83         //3a. Lấy phần tử ở đầu hàng đợi
84         ElementType pair = front(&Q); dequeue(&Q);
85         int u = pair.u, p = pair.p;
86         if (mark[u] != 0) //u đã duyệt rồi, bỏ qua
87             continue;
88
89         //printf("Duyệt %d\n", u); //làm gì đó trên u

```

```

90 //printf("Đuyệt xu(n, u); //Lưu gì đó trên u
91 mark[u] = 1; //Đánh dấu nó đã duyệt
92 parent[u] = p; //Đánh dấu nó đã duyệt
93
94 //3b. Xét các đỉnh kề của u, đưa vào hàng đợi Q
95 for (int v = 1; v <= pG->n; v++)
96     if (adjacent(pG, u, v)) {
97         ElementType pair;
98         pair.u = v; pair.p = u;
99         enqueue(&Q, pair);
100     }
101 }
102
103
104
105 int main() {
106     //1. Khai báo đồ thị G
107     Graph G;
108     //2. Đọc dữ liệu và dựng đồ thị
109     int n, m, u, v;
110     scanf("%d%d", &n, &m);
111     init_graph(&G, n);
112     for (int e = 0; e < m; e++) {
113         scanf("%d%d", &u, &v);
114         add_edge(&G, u, v);
115     }
116
117     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
118     for (int u = 1; u <= G.n; u++) {
119         mark[u] = 0;
120         parent[u] = -1;
121     }
122
123     //4. Gọi hàm DFS
124     for (int u = 1; u <= G.n; u++)
125         if (mark[u] == 0)
126             BFS(&G, u);
127
128
129     for (int u = 1; u <= G.n; u++)
130         printf("%d %d\n", u, parent[u]);
131
132
133     return 0;
134 }
135
136

```

Correct

Marks for this submission: 1.00/1.00.



Cho một đồ thị **vô hướng đơn**. Hãy duyệt (các) cây DUYỆT ĐỒ THỊ khi duyệt đồ thị theo chiều sâu bắt đầu từ đỉnh 1.

Nếu vẫn còn đỉnh chưa duyệt sau khi duyệt xong lần 1, tìm đỉnh có chỉ số nhỏ nhất chưa duyệt mà duyệt nó, và cứ tiếp tục như thế cho đến khi tất cả các đỉnh đều được duyệt.

Quy ước:

- Các đỉnh kề của 1 đỉnh được liệt kê *theo thứ tự tăng dần*
- Nếu cài đặt dùng ngăn xếp hãy đưa các đỉnh kề của đỉnh u vào ngăn xếp theo **thứ tự từ lớn đến nhỏ**

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m, tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v).

Đầu ra (Output)

- In ra cây duyệt đồ thị theo định dạng:

```
1 <đỉnh cha của 1>
2 <đỉnh cha của 2>
....
i <đỉnh cha của i>
...
n <đỉnh cha của n>
```

[Copy to BlackBox](#)

- Nếu 1 đỉnh không có đỉnh cha (nó là đỉnh gốc của cây) thì đỉnh cha của nó là -1.
- Xem thêm các ví dụ bên dưới.

Gợi ý

- Sử dụng mảng parent[u] để lưu đỉnh cha của đỉnh u.
- Trong quá trình duyệt, thay vì in các đỉnh ra màn hình, ghi nhận lại đỉnh cha của các đỉnh.
- Khi duyệt xong lần lượt in ra u và parent[u] (u chạy từ 1 đến n).

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

[Copy to BlackBox](#)

For example:

Input	Result
4 3	1 -1
2 1	2 1
1 4	3 -1
2 4	4 2

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct{
```



```

8     int n,m;
9     int A[MAX_SIZE][MAX_SIZE];
10 }Graph;
11
12 //init graph with n vertex
13 void init_graph(Graph* pG, ElementType n){
14     pG->n = n;
15     pG->m = 0;
16
17     //cho toan bo ma tran ke bang 0
18     for(int i = 0; i <= pG->n; i++){
19         for(int j = 0; j <= pG->n; j++){
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 //add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v){
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 int visited[MAX_SIZE];
34 int parent[MAX_SIZE];
35
36 void DFS (Graph* pG, int s, int p){
37     visited[s] = 1;
38     // printf("%d \n", s);
39     parent[s] = p;
40     for(int u = 1; u <= pG->n; u++){
41         if(!visited[u] && pG->A[s][u] != 0 ){
42             DFS(pG,u,s);
43         }
44     }
45 }
46
47 int main(){
48     Graph G;
49     int n, m, u, v, e;
50     scanf("%d%d", &n, &m);
51     init_graph(&G, n);
52
53     for (e = 0; e < m; e++) {
54         scanf("%d%d", &u, &v);
55         add_edge(&G, u, v);
56     }
57
58     for(int i = 0; i <= G.n; i++){
59         visited[i] = 0;
60         parent[i] = -1;
61     }
62
63     for(int i = 1; i <= G.n; i++){
64         if(!visited[i]){
65             DFS(&G,i,-1);
66         }
67     }
68
69     for(int i = 1; i <= G.n; i++){
70         printf("%d %d\n", i, parent[i]);
71     }
72 }

```

	Input	Expected	Got	
✓	4 3	1 -1	1 -1	✓
	2 1	2 1	2 1	
	1 4	3 -1	3 -1	
	2 4	4 2	4 2	

	Input	Expected	Got	
✓	4 2 1 2 3 4	1 -1 2 1 3 -1 4 3	1 -1 2 1 3 -1 4 3	✓
✓	4 2 1 4 2 3	1 -1 2 -1 3 2 4 1	1 -1 2 -1 3 2 4 1	✓
✓	13 15 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	1 -1 2 1 3 7 4 2 5 -1 6 4 7 6 8 5 9 8 10 12 11 10 12 1 13 6	1 -1 2 1 3 7 4 2 5 -1 6 4 7 6 8 5 9 8 10 12 11 10 12 1 13 6	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35 int parent[MAX_N];
36
37 void DFS(Graph *pG, int u, int p) {
38     //1. Đánh dấu u đã duyệt
39     //printf("Duyet %d\n", u); //Làm gì đó trên u
40     mark[u] = 1; //Đánh dấu nó đã duyệt
41     parent[u] = p; //Cho cha của u là p
42
43     //2. Xét các đỉnh kề của u

```

```

43 //2. Xét các đỉnh kề của u
44 for (int v = 1; v <= pG->n; v++)
45     if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
46         DFS(pG, v, u); //gọi đệ quy duyệt nó
47
48 }
49
50
51
52
53 int main() {
54     //1. Khai báo đồ thị G
55     Graph G;
56     //2. Đọc dữ liệu và dựng đồ thị
57     int n, m, u, v;
58     scanf("%d%d", &n, &m);
59     init_graph(&G, n);
60     for (int e = 0; e < m; e++) {
61         scanf("%d%d", &u, &v);
62         add_edge(&G, u, v);
63     }
64
65     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
66     for (int u = 1; u <= G.n; u++) {
67         mark[u] = 0;
68         parent[u] = -1;
69     }
70
71     //4. Gọi hàm DFS
72     for (int u = 1; u <= G.n; u++)
73         if (mark[u] == 0)
74             DFS(&G, u, -1);
75
76
77     for (int u = 1; u <= G.n; u++)
78         printf("%d %d\n", u, parent[u]);
79
80
81     return 0;
82 }
83
84

```

Correct

Marks for this submission: 1.00/1.00. Accounting for previous tries, this gives **0.90/1.00**.



Cho một đồ thị **có hướng đơn**. Hãy dựng (các) cây DUYỆT ĐỒ THỊ khi duyệt đồ thị theo chiều sâu bắt đầu từ đỉnh 1.

Nếu vẫn còn đỉnh chưa duyệt sau khi duyệt xong lần 1, tìm đỉnh có chỉ số nhỏ nhất chưa duyệt mà duyệt nó, và cứ tiếp tục như thế cho đến khi tất cả các đỉnh đều được duyệt.

Quy ước:

- Các đỉnh kề của 1 đỉnh được liệt kê *theo thứ tự tăng dần*
- Nếu cài đặt dùng ngăn xếp hãy đưa các đỉnh kề của đỉnh u vào ngăn xếp theo **thứ tự từ lớn đến nhỏ**

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m, tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v).

Đầu ra (Output)

- In ra cây duyệt đồ thị theo định dạng:

```
1 <đỉnh cha của 1>
2 <đỉnh cha của 2>
....
i <đỉnh cha của i>
...
n <đỉnh cha của n>
```

[Copy to BlackBox](#)

- Nếu 1 đỉnh không có đỉnh cha (nó là đỉnh gốc của cây) thì đỉnh cha của nó là -1.
- Xem thêm các ví dụ bên dưới.

Gợi ý

- Sử dụng mảng parent[u] để lưu đỉnh cha của đỉnh u.
- Trong quá trình duyệt, thay vì in các đỉnh ra màn hình, ghi nhận lại đỉnh cha của các đỉnh.
- Khi duyệt xong lần lượt in ra u và parent[u] (u chạy từ 1 đến n).

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

[Copy to BlackBox](#)

For example:

Input	Result
4 3	1 -1
2 1	2 -1
1 4	3 -1
2 4	4 1

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct{
```



```

8     int n,m;
9     int A[MAX_SIZE][MAX_SIZE];
10 }Graph;
11
12 //init graph with n vertex
13 void init_graph(Graph* pG, ElementType n){
14     pG->n = n;
15     pG->m = 0;
16
17     //cho toan bo ma tran ke bang 0
18     for(int i = 0; i <= pG->n; i++){
19         for(int j = 0; j <= pG->n; j++){
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 //add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v){
27     pG->A[u][v] = 1;
28     // pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 int visited[MAX_SIZE];
34 int parent[MAX_SIZE];
35
36 void DFS (Graph* pG, int s, int p){
37     visited[s] = 1;
38     // printf("%d \n", s);
39     parent[s] = p;
40     for(int u = 1; u <= pG->n; u++){
41         if(!visited[u] && pG->A[s][u] != 0 ){
42             DFS(pG,u,s);
43         }
44     }
45 }
46
47 int main(){
48     Graph G;
49     int n, m, u, v, e;
50     scanf("%d%d", &n, &m);
51     init_graph(&G, n);
52
53     for (e = 0; e < m; e++) {
54         scanf("%d%d", &u, &v);
55         add_edge(&G, u, v);
56     }
57
58     for(int i = 0; i <= G.n; i++){
59         visited[i] = 0;
60         parent[i] = -1;
61     }
62
63     for(int i = 1; i <= G.n; i++){
64         if(!visited[i]){
65             DFS(&G,i,-1);
66         }
67     }
68
69     for(int i = 1; i <= G.n; i++){
70         printf("%d %d\n", i, parent[i]);
71     }
72 }

```

	Input	Expected	Got	
✓	4 3	1 -1	1 -1	✓
	2 1	2 -1	2 -1	
	1 4	3 -1	3 -1	
	2 4	4 1	4 1	

	Input	Expected	Got	
✓	4 2 1 2 3 4	1 -1 2 1 3 -1 4 3	1 -1 2 1 3 -1 4 3	✓
✓	4 2 1 4 2 3	1 -1 2 -1 3 2 4 1	1 -1 2 -1 3 2 4 1	✓
✓	13 15 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	1 -1 2 1 3 -1 4 2 5 -1 6 4 7 6 8 5 9 8 10 -1 11 10 12 1 13 6	1 -1 2 1 3 -1 4 2 5 -1 6 4 7 6 8 5 9 8 10 -1 11 10 12 1 13 6	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     //if (u != v)
21     //    pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35 int parent[MAX_N];
36
37 void DFS(Graph *pG, int u, int p) {
38     //1. Đánh dấu u đã duyệt
39     //printf("Duyet %d\n", u); //Làm gì đó trên u
40     mark[u] = 1; //Đánh dấu nó đã duyệt
41     parent[u] = p; //Cho cha của u là p
42
43     //2. Xét các đỉnh kề của u

```

```

43 //2. Xét các đỉnh kề của u
44 for (int v = 1; v <= pG->n; v++)
45     if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
46         DFS(pG, v, u); //gọi đệ quy duyệt nó
47
48 }
49
50
51
52
53 int main() {
54     //1. Khai báo đồ thị G
55     Graph G;
56     //2. Đọc dữ liệu và dựng đồ thị
57     int n, m, u, v;
58     scanf("%d%d", &n, &m);
59     init_graph(&G, n);
60     for (int e = 0; e < m; e++) {
61         scanf("%d%d", &u, &v);
62         add_edge(&G, u, v);
63     }
64
65     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
66     for (int u = 1; u <= G.n; u++) {
67         mark[u] = 0;
68         parent[u] = -1;
69     }
70
71     //4. Gọi hàm DFS
72     for (int u = 1; u <= G.n; u++)
73         if (mark[u] == 0)
74             DFS(&G, u, -1);
75
76     for (int u = 1; u <= G.n; u++)
77         printf("%d %d\n", u, parent[u]);
78
79
80
81     return 0;
82 }
83
84

```

Correct

Marks for this submission: 1.00/1.00.

◀ Bài 6 - Đọc đồ thị từ tập tin

Jump to...

* Bài tập 7. Kiểm tra đồ thị vô hướng liên thông ▶

